TABLE OF CONETENTS

Seriel number	Title	Page.no
1	ABSTRACT	2
2	REQUIREMENT ANALYSIS	4
3	APPLICATION	5
4	DESIGN	7
5	FLOW CHART	9
6	ALGORITHM	11
7	SOURCE CODE	14
8	EXECUTION SCREEN SHOTS/SAMPLE OUTPUTS	25
9	CONCLUSION	33
10	FUTURE ENHANCEMENT	34
11	BIBLIOGRAPHY / REFERENCES	35

ABSTRACT

In the ever-evolving landscape of innovation and intellectual property, efficient management and tracking of patents is crucial for both inventors and organizations. This project, titled "Patent Management System with SQL Integration", is designed to serve as a dynamic, scalable, and user-friendly software solution for storing, retrieving, and analyzing patent data. Initially implemented using in-memory storage with Python, the system has been significantly enhanced with the integration of an SQL-based backend to ensure persistent data storage, better scalability, and real-time data manipulation capabilities.

The core objective of this project is to provide users with a seamless interface for adding, searching, filtering, sorting, and analyzing patent records. The transition from a volatile memory model to a structured SQL database eliminates data loss upon program termination and allows for more complex querying and data analytics. Patent records consist of a unique ID, title, inventor name, filing date, and a brief description — all stored securely within the database.

Users can perform keyword-based searches across titles and descriptions, filter patents by year or inventor, sort entries alphabetically or chronologically, and view insightful statistics such as patent counts by year and inventor. The program is designed using modular logic, encouraging future scalability — such as web deployment, user authentication, or cloud database integration.

This system is particularly relevant in educational settings for demonstrating database connectivity, CRUD operations, and basic data analytics. It also lays a foundational blueprint for real-world intellectual property systems. By uniting Python's intuitive interface with SQL's robust data handling, the Patent Management System bridges the gap between conceptual innovation and practical implementation — a tool that does not just record invention, but celebrates it.

REQUIREMENT ANALYSIS

i) HARDWARE REQUIREMENTS :

Processor (CPU)	Intel Pentium Dual Core / AMD equivalent	Intel i3 or higher
RAM	2 GB	4 GB or more
Storage	500 MB free disk space	1 GB free for future extensions
Display	1024 x 768 resolution	1366 x 768 or higher
Keyboard & Mouse	Standard input devices	Standard input devices
Operating System	Windows 7/8/10/11, Linux, or macOS	Windows 10/11 or Ubuntu 20.04+

ii) SOFTWARE REQUIREMENTS :

Operating System	Windows 7/8/10/11, Linux (Ubuntu/Fedora), or macOS
Programming Language	Python 3.8 or higher
Database	Mysql (Built-in with Python's mysql module)
IDE / Code Editor	Any (Recommended: VS Code / PyCharm / IDLE python)
Python Libraries Used	mysql, datetime, os (All are standard Python libraries)
Command Line Interface	Windows Command Prompt

APPLICATION

The Patent Management System is a simplified, command-line-based tool designed to manage and maintain records of patents efficiently. It serves as a foundational model for real-world patent databases and IP (Intellectual Property) management tools used by law firms, inventors, companies, and patent offices.

Educational Use:

•Ideal for computer science students learning about data handling, file management, and databases.

•Demonstrates CRUD operations, sorting, searching, filtering, and generating statistics — all vital programming concepts.

Patent Record Management:

•Can be used by individual inventors or small R&D teams to keep track of their innovations.

•Tracks essential details such as patent ID, title, inventor, filing date, and description.

Research and Innovation Cells:

•Useful for internal IP cells in academic institutions to catalog student or faculty innovations.

•Can serve as a prototype for integrating more complex systems like patent application tracking or renewal alerts.

Backend Practice for Developers:

•A strong base for students exploring backend development using Python and SQL.

•Can be extended to support web-based dashboards, secure logins, and real-time analytics.

Academic and Research Institutes:

Used by universities, colleges, and research centers to store, search, and manage details of new inventions and patents filed by faculty and students. It helps track intellectual contributions, avoid missed renewal deadlines, and promotes innovation.evalueserve

Corporate R&D Departments:

Companies use patent management systems to manage large portfolios, search for patents by status or region, monitor expiry dates, and support licensing and protection of inventions. It is crucial for ensuring ideas are protected and legal time-frames are met, supporting overall business strategy.linkedin+1

Legal and Compliance Teams:

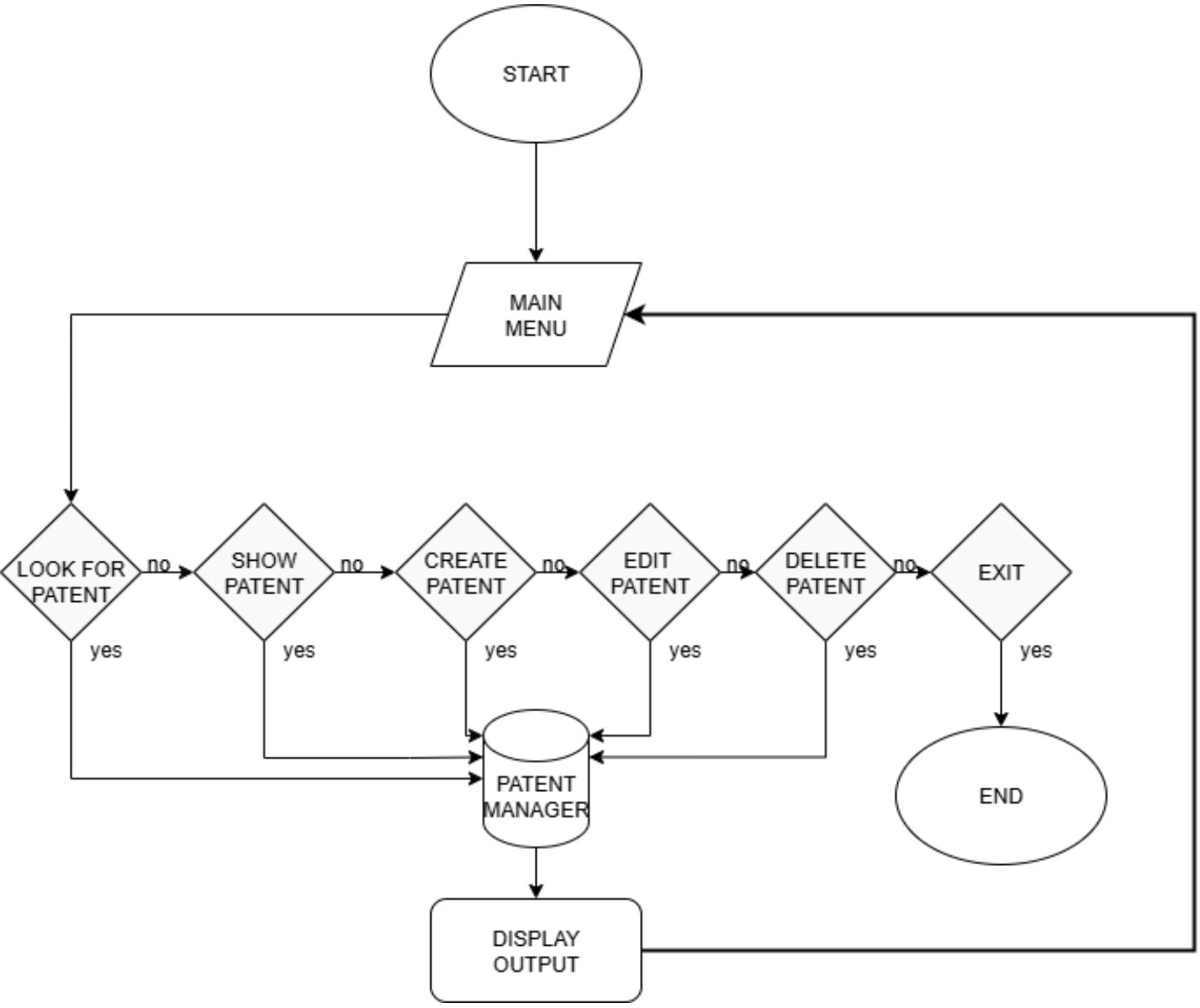
Law firms and patent attorneys use these systems for prosecution support, renewal management, enforcement actions, and to avoid legal pitfalls. Helps in tracking patent litigation risks and making faster decisions for clients.computerpackages+1

Government Agencies:

Patent offices and government IP departments utilize such platforms to organize national patent databases, facilitate patent searches by the public, and handle patent filings, grants, and disputes efficiently.

DESIGN

i) BLOCK DIAGRAM:-



i) EXPLANATION :-

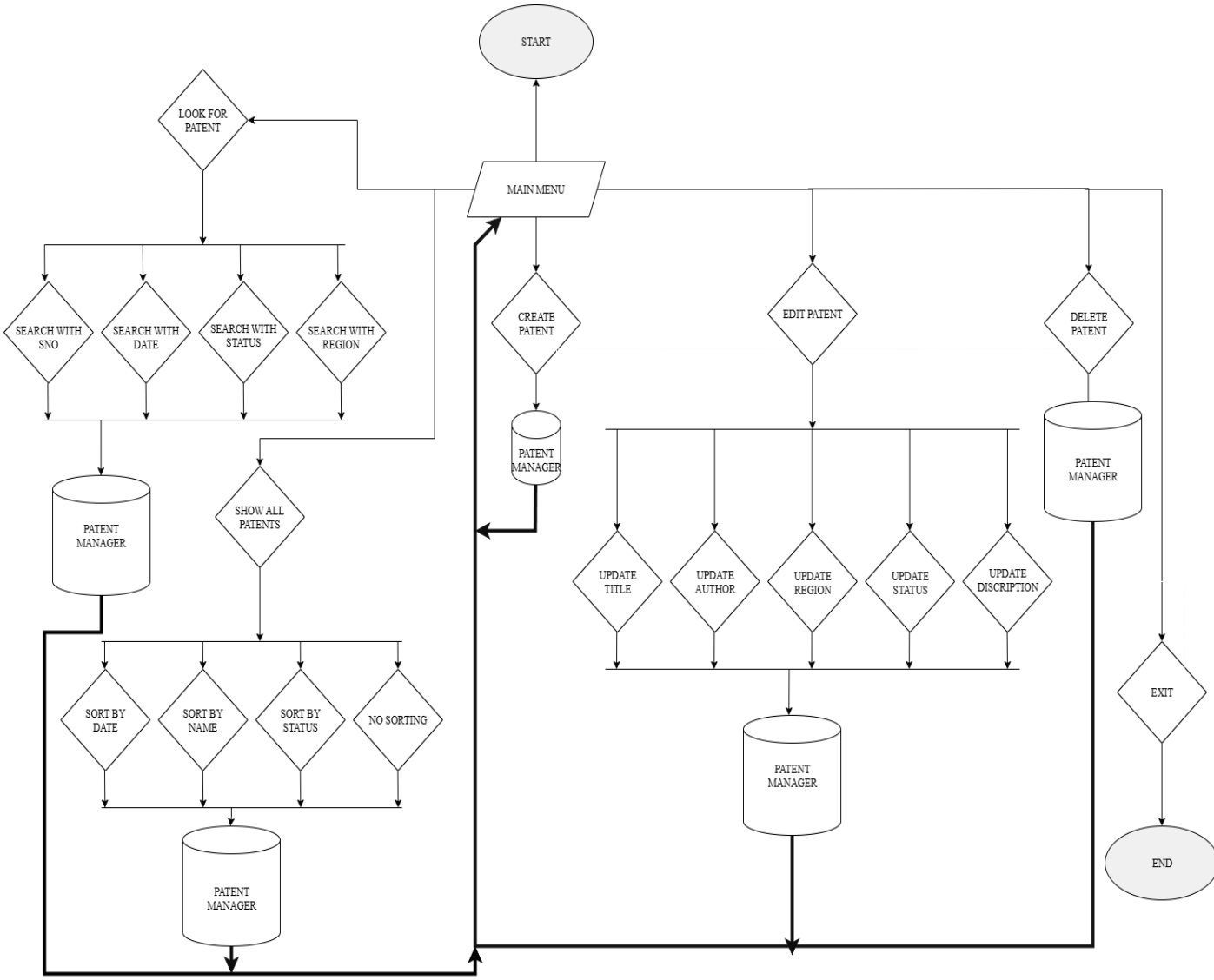
The block diagram of the Patent Management System provides a structured overview of how the application operates. The system flow begins at the "Start" point (represented by an oval), signifying the launch of the program. It then moves to the "Main Menu" block, which is the central point of user interaction. The main menu presents options such as looking for a patent, showing all patents, creating, editing, deleting records, and exiting the application.

Each option from the menu is depicted by a decision diamond, guiding the flow to different action blocks based on the user's choice. For example, if the user chooses to look for a patent, the system navigates to the search functionality; if the user wants to create a new patent, it leads to the data entry process. Similarly, there are dedicated blocks for editing or deleting patent records. All these operations interact with the "Patent Manager" block, illustrated as a cylinder, representing the MySQL database where all patent records are stored securely.

After any action—such as adding, searching, or deleting a patent—the results or information are sent to the “Display Output” block. This block shows output or confirmation messages to the user and then brings control back to the main menu. The entire program loops through these processes, ensuring the user can perform multiple operations without restarting the system. Finally, if the exit option is selected, the flow moves to the "End" block, gracefully terminating the program.

Overall, the block diagram clearly visualizes the modular structure and cyclic workflow of the project. It emphasizes the sequence of user choices, processing steps, and database management, making it easy to understand for both project reviewers and users.

FLOW CHART


START
MAIN MENU
LOOK FOR PATENT
SEARCH WITH SNO
SEARCH WITH DATE
SEARCH WITH STATUS
SEARCH WITH REGION
PATENT MANAGER
SHOW ALL PATENTS
SORT BY DATE
SORT BY NAME
SORT BY STATUS
NO SORTING
PATENT MANAGER
CREATE PATENT
PATENT MANAGER
EDIT PATENT
UPDATE TITLE
UPDATE AUTHOR
UPDATE REGION
UPDATE STATUS
UPDATE DISCRIPTION
PATENT MANAGER
DELETE PATENT
PATENT MANAGER
EXIT
END

EXPLANATION :

The flow chart of the Patent Management System offers a step-by-step visualization of the program's logic and menu-driven workflow. Starting from the "Start" oval, the system immediately presents the user with the "Main Menu." This menu functions as a central navigation panel, allowing users to select from various options, such as searching for patents, adding new records, editing existing patents, deleting patents, and viewing all stored records.

Each menu selection is illustrated as a diamond-shaped decision node. Based on the user's input, the flow chart branches into separate logical paths. Selecting "Look For Patent" leads to a further breakdown through sub-decisions, allowing searches by serial number, date, status, or region. Each search type directs the system to access the database, fetch relevant results, and then display them to the user before looping back to the main menu.

Choosing "Show All Patents" opens a new set of decisions, enabling the records to be sorted by date, name, status, or shown without sorting. These choices determine the SQL query used, display the appropriate list, and then return control to the user.

The "Create Patent" path asks the user to input all required details before saving the new record to the central database. The "Edit Patent" path involves a sequence of sub-decisions, each handling the update of a specific field such as the title, author, region, status, or description. Each edit operation is reflected in the database, and confirmation is provided on completion.

The "Delete Patent" option prompts the user for confirmation before removing the selected record from the database to prevent accidental data loss. Every successful action flows back to the main menu, ensuring repeated use in a continuous session.

Finally, if the user selects the "Exit" option, the program transitions to the "End" oval, finishing the session gracefully.

ALGORITHM

Algorithm for Patent Management System

1. Start the program.
2. Establish a connection to the MySQL database.
3. Display the Main Menu with options:
   - Look for Patent
   - Show All Patents
   - Create Patent
   - Edit Patent
   - Delete Patent
   - Exit
4. Prompt the user to enter their menu choice.

5. If the choice is "Look for Patent":
   a. Display sub-menu: Search by Serial Number, Date, Status, or Region.
   b. Take the user’s search input.
   c. Query the database for matching records.
   d. If records are found, display results; else show "No patent found".
   e. Return to main menu.

6. If the choice is "Show All Patents":
   a. Display sorting options: by Date, Name, Status, or no sorting.
   b. Take the user’s choice and query all rows accordingly.
   c. Display the resulting list of patents.
   d. Return to main menu.

7. If the choice is "Create Patent":
   a. Prompt for patent details (title, author, region, status, description).
   b. Generate next serial number and get current date/time.
   c. Insert the new record into the database.
   d. Show confirmation message.
   e. Return to main menu.

8. If the choice is "Edit Patent":
   a. Ask for the serial number.
   b. Fetch and display current patent details.
   c. Show options for editing fields (title, author, region, status, description).
   d. Take user input for selected field and new value.
   e. Update the database record.
   f. Show success or failure message.
   g. Return to main menu.

9. If the choice is "Delete Patent":
   a. Prompt for the serial number.
   b. Show current details for confirmation.
   c. If confirmed, delete the record from the database.
   d. Show success or cancel message.
   e. Return to main menu.

10. If the choice is "Exit":
   a. Close the database connection.
   b. Terminate the program.

11. Repeat from step 4 until the user chooses to exit.

SOURCE CODE

PYTHON SOURCE CODE :

```
#PATENT MANAGER

import mysql.connector

import datetime


def con():
    conn = mysql.connector.connect(
        host='localhost',
        user='root',
        password='2025',
        database='patent_manager'
    )
    return conn



#1
def search_by_sno():

    conn = con()
    cur=conn.cursor()

    sno = int(input("Enter seriel number to search: "))

    cur.execute("select * from patents where s_no = %s", (sno,))
    result = cur.fetchone()

    if result:
        print(result)
        print("-" * 50)
```

```
    else:
        print("No patent found!")


    cur.close()
    conn.close()


def search_by_date():
    conn = con()
    cur=conn.cursor()
    date = input("enter date (YYYY-MM-DD): ")

    cur.execute("select * from patents where date(date_time) = %s", (date,))
    results = cur.fetchall()

    if results:
        for i in results:
            print(i)
            print("-" * 50)


    else:
        print("No patents found for this date!")

    cur.close()
    conn.close()

def search_by_status():
    print("\n--- Search by Status  ---")
    status = input("Enter status to search: ")
    conn = con()
    cur = conn.cursor()
    cur.execute("select * from patents where status = %s", (status,))
```

```
    results = cur.fetchall()
    if results:
        for i in results:
            print(i)
            print("-" * 50)
    else:
        print("No patents found for the given criteria.")
    cur.close()
    conn.close()


def search_by_region():
    print("\n--- Search by Region  ---")
    region = input("Enter region to search: ")
    conn = con()
    cur = conn.cursor()
    cur.execute("select * from patents WHERE region = %s", (region,))
    results = cur.fetchall()
    if results:
        for i in results:
            print(i)
            print("-" * 50)
    else:
        print("No patents found for this region.")
    cur.close()
    conn.close()
```

```
def look_for_patent():
    print("\n--- Search Patent ---")
    print("1. Search by S_No")
    print("2. Search by Date")
    print("3. Search by status")
    print("4. Search by region")

    choice = int(input("Enter your choice : "))

    if choice == 1:
        search_by_sno()
    elif choice == 2:
        search_by_date()
    elif choice ==3:
        search_by_status()
    elif choice ==4:
        search_by_region()
    else:
        print('wrong option choosed')




#2
def show_all_patents():

    print("\n--- Show All Patents ---")
    print("1. Sort by Date")
    print("2. Sort by Name")
    print("3. Sort by Status")
    print("4. Show without sorting")
```

```
choice = input("Enter your choice (1-4): ")

conn = con()
cur=conn.cursor()


if choice == '1':
    cur.execute("select * from patents order by date_time")

elif choice == '2':
    cur.execute("select * from patents order by author_name")

elif choice == '3':
    cur.execute("select * from patents order by status")

else:
    cur.execute("select * from patents")

results = cur.fetchall()

for i in results:
    print(i)
    print("-" * 50)


cur.close()
conn.close()
```

```
#3

def sno():

    conn = con()
    cur=conn.cursor()

    cur.execute("select max(s_no) from patents")
    result = cur.fetchone()

    next_sno = 1 if result[0] is None else result[0] + 1
    cur.close()
    conn.close()
    return next_sno

def create_patent():
    print("\n--- Create New Patent ---")

    s_no =sno()
    print(f"S_No: {s_no}")

    title = input("Enter title: ")
    author_name = input("Enter author name: ")
    region = input("Enter region: ")
    status = input("Enter status: ")
    idea_description = input("Enter description: ")

    date_time = datetime.datetime.now()

    conn = con()
    cur=conn.cursor()
```

```
    query = """
    insert into patents (s_no, title, author_name, date_time, region, status, idea_description)
    VALUES (%s, %s, %s, %s, %s, %s, %s)
    """
    values = (s_no, title, author_name, date_time, region, status, idea_description)

    cur.execute(query, values)
    conn.commit()
    print("Patent created successfully!")

    cur.close()
    conn.close()

#4
def edit_patent():
    print("\n--- Edit Patent ---")

    s_no = input("Enter Seriel number to edit: ")

    conn = con()
    cur=conn.cursor()

    cur.execute("select * from patents where s_no = %s", (s_no,))
    result = cur.fetchone()

    if  not result :
        print("Patent not found!")
        cur.close()
        conn.close()
        return

    print("Current details:")
    print(result)
```

```
print("\n1. Update Title")
print("2. Update Author")
print("3. Update Region")
print("4. Update Status")
print("5. Update Description")

choice = input("choose an option from above: ")

if choice == '1':
    new_value = input("New title: ")
    cur.execute("update patents set title = %s where s_no = %s", (new_value, s_no))

elif choice == '2':
    new_value = input("New author: ")
    cur.execute("update patents set author_name = %s where s_no = %s", (new_value, s_no))

elif choice == '3':
    new_value = input("New region: ")
    cur.execute("update patents set region = %s where s_no = %s", (new_value, s_no))

elif choice == '4':
    new_value = input("New status: ")
    cur.execute("update patents set status = %s where s_no = %s", (new_value, s_no))

elif choice == '5':
    new_value = input("New description: ")
    cur.execute("update patents set idea_description = %s where s_no = %s", (new_value, s_no))

conn.commit()
print("Updated successfully!")

cur.close()
conn.close()
```

```python
#5
def delete_patent():
    print("\n--- Delete Patent ---")
    s_no = int(input("Enter Serial number to delete: "))
    conn = con()
    cur = conn.cursor()

    cur.execute("select * from patents where s_no = %s", (s_no,))
    result = cur.fetchone()

    if not result:
        print("Patent not found!")
    else:
        confirm = input("Are you sure you want to delete patent ? (y/n): ")
        if confirm.lower() == 'y':
            cur.execute("delete from patents where s_no = %s", (s_no,))
            conn.commit()
            print("Patent deleted successfully!")
        else:
            print("Deletion cancelled.")

    cur.close()
    conn.close()
```

```
while True:

    print("\n--- PATENT MANAGEMENT SYSTEM ---")
    print("1. Look for a Patent")
    print("2. Show All Patents")
    print("3. Create Patent")
    print("4. Edit Patent")
    print("5.Delete patent")
    print("6. Exit")

    choice = input("Choose any one option from above: ")

    if choice == '1':
      look_for_patent()
    elif choice == '2':
       show_all_patents()
    elif choice == '3':
       create_patent()
    elif choice == '4':
       edit_patent()
    elif choice =='5':
       delete_patent()
    elif choice == '6':
       break
```

MYSQL SOURCE CODE :

```
mysql> use patent_manager
Database changed
```

```
mysql> desc patents;
+------------------+--------------+------+-----+---------+-------+
| Field            | Type         | Null | Key | Default | Extra |
+------------------+--------------+------+-----+---------+-------+
| s_no             | int          | NO   | PRI | NULL    |       |
| title            | varchar(255) | NO   |     | NULL    |       |
| author_name      | varchar(100) | NO   |     | NULL    |       |
| date_time        | datetime     | NO   |     | NULL    |       |
| region           | varchar(75)  | YES  |     | NULL    |       |
| status           | varchar(50)  | NO   |     | NULL    |       |
| idea_description | text         | YES  |     | NULL    |       |
+------------------+--------------+------+-----+---------+-------+
7 rows in set (0.08 sec)
```

EXECUTION SCREEN SHOTS/SAMPLE OUTPUTS

MY SQL TABLE :

```
mysql> select*from patents
    -> ;
+------+---------+-------------+---------------------+----------+-----------+-----------------+
| s_no | title   | author_name | date_time           | region   | status    | idea_description |
+------+---------+-------------+---------------------+----------+-----------+-----------------+
|    1 | title 1 | author 1    | 2025-10-08 20:59:20 | reg 1    | pending   | description1     |
|    2 | title 2 | author 2    | 2025-10-08 20:59:49 | region 2 | completed | discription 2    |
|    3 | title 3 | author 3    | 2025-10-08 21:00:19 | region 3 | completed | discription 3    |
|    4 | title 4 | author 4    | 2025-10-08 21:00:36 | region 4 | pending   | discription 4    |
+------+---------+-------------+---------------------+----------+-----------+-----------------+
4 rows in set (0.02 sec)
```

- To look for a patent using seriel number

```
--- PATENT MANAGEMENT SYSTEM ---
1. Look for a Patent
2. Show All Patents
3. Create Patent
4. Edit Patent
5.Delete patent
6. Exit
Choose any one option from above: 1

--- Search Patent ---
1. Search by S_No
2. Search by Date
3. Search by status
4. Search by region
Enter your choice : 1
Enter seriel number to search: 1
(1, 'title 1', 'author 1', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'description
1')
--------------------------------------------------
```

- To look for a patent using date

```
--- Search Patent ---
1. Search by S_No
2. Search by Date
3. Search by status
4. Search by region
Enter your choice : 2
enter date (YYYY-MM-DD): 2025-10-08
(1, 'title 1', 'author 1', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'description
1')
--------------------------------------------------
(2, 'title 2', 'author 2', datetime.datetime(2025, 10, 8, 20, 59, 49), 'region 2', 'completed', 'discr
iption 2')
--------------------------------------------------
(3, 'title 3', 'author 3', datetime.datetime(2025, 10, 8, 21, 0, 19), 'region 3', 'completed', 'discrip
tion 3')
--------------------------------------------------
(4, 'title 4', 'author 4', datetime.datetime(2025, 10, 8, 21, 0, 36), 'region 4', 'pending', 'discripti
on 4')
--------------------------------------------------
```

- To look for a patent using Status

```
--- Search Patent ---
1. Search by S_No
2. Search by Date
3. Search by status
4. Search by region
Enter your choice : 3

--- Search by Status  ---
Enter status to search: pending
(1, 'title 1', 'author 1', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'description
1')
--------------------------------------------------
(4, 'title 4', 'author 4', datetime.datetime(2025, 10, 8, 21, 0, 36), 'region 4', 'pending', 'discripti
on 4')
--------------------------------------------------
```

- To look for a patent using Region

```
--- Search Patent ---
1. Search by S_No
2. Search by Date
3. Search by status
4. Search by region
Enter your choice : 4

--- Search by Region  ---
Enter region to search: reg 1
(1, 'title 1', 'author 1', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'description
1')
--------------------------------------------------
```

- To show all patents sorting by date

```
--- PATENT MANAGEMENT SYSTEM ---
1. Look for a Patent
2. Show All Patents
3. Create Patent
4. Edit Patent
5.Delete patent
6. Exit
Choose any one option from above: 2

--- Show All Patents ---
1. Sort by Date
2. Sort by Name
3. Sort by Status
4. Show without sorting
Enter your choice (1-4): 1
(1, 'title 1', 'author 1', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'description
1')
--------------------------------------------------
(2, 'title 2', 'author 2', datetime.datetime(2025, 10, 8, 20, 59, 49), 'region 2', 'completed', 'discr
iption 2')
--------------------------------------------------
(3, 'title 3', 'author 3', datetime.datetime(2025, 10, 8, 21, 0, 19), 'region 3', 'completed', 'discrip
tion 3')
--------------------------------------------------
(4, 'title 4', 'author 4', datetime.datetime(2025, 10, 8, 21, 0, 36), 'region 4', 'pending', 'discripti
on 4')
--------------------------------------------------
```

- To show all patents sorting by name

```
--- Show All Patents ---
1. Sort by Date
2. Sort by Name
3. Sort by Status
4. Show without sorting
Enter your choice (1-4): 2
(1, 'title 1', 'author 1', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'description
1')
--------------------------------------------------
(2, 'title 2', 'author 2', datetime.datetime(2025, 10, 8, 20, 59, 49), 'region 2', 'completed', 'discr
iption 2')
--------------------------------------------------
(3, 'title 3', 'author 3', datetime.datetime(2025, 10, 8, 21, 0, 19), 'region 3', 'completed', 'discrip
tion 3')
--------------------------------------------------
(4, 'title 4', 'author 4', datetime.datetime(2025, 10, 8, 21, 0, 36), 'region 4', 'pending', 'discripti
on 4')
--------------------------------------------------
```

- To show all patents sorting by Status

```
--- Show All Patents ---
1. Sort by Date
2. Sort by Name
3. Sort by Status
4. Show without sorting
Enter your choice (1-4): 3
(2, 'title 2', 'author 2', datetime.datetime(2025, 10, 8, 20, 59, 49), 'region 2', 'completed', 'discr
iption 2')
--------------------------------------------------
(3, 'title 3', 'author 3', datetime.datetime(2025, 10, 8, 21, 0, 19), 'region 3', 'completed', 'discrip
tion 3')
--------------------------------------------------
(1, 'title 1', 'author 1', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'description
1')
--------------------------------------------------
(4, 'title 4', 'author 4', datetime.datetime(2025, 10, 8, 21, 0, 36), 'region 4', 'pending', 'discripti
on 4')
--------------------------------------------------
```

- To show all patents without sorting

```
--- Show All Patents ---
1. Sort by Date
2. Sort by Name
3. Sort by Status
4. Show without sorting
Enter your choice (1-4): 4
(1, 'title 1', 'author 1', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'description
1')
--------------------------------------------------
(2, 'title 2', 'author 2', datetime.datetime(2025, 10, 8, 20, 59, 49), 'region 2', 'completed', 'discr
iption 2')
--------------------------------------------------
(3, 'title 3', 'author 3', datetime.datetime(2025, 10, 8, 21, 0, 19), 'region 3', 'completed', 'discrip
tion 3')
--------------------------------------------------
(4, 'title 4', 'author 4', datetime.datetime(2025, 10, 8, 21, 0, 36), 'region 4', 'pending', 'discripti
on 4')
--------------------------------------------------
```

- To Create a patent with user inputs (Dynamically)

```
--- PATENT MANAGEMENT SYSTEM ---
1. Look for a Patent
2. Show All Patents
3. Create Patent
4. Edit Patent
5.Delete patent
6. Exit
Choose any one option from above: 3

--- Create New Patent ---
S_No: 5
Enter title: title 5
Enter author name: author 5
Enter region: region 5
Enter status: completed
Enter description: discription 5
Patent created successfully!
```

- After creating a new patent changes made in the database

```
mysql> select*from patents
    -> ;
+------+---------+-------------+---------------------+----------+-----------+------------------+
| s_no | title   | author_name | date_time           | region   | status    | idea_description |
+------+---------+-------------+---------------------+----------+-----------+------------------+
|    1 | title 1 | author 1    | 2025-10-08 20:59:20 | reg 1    | pending   | description1     |
|    2 | title 2 | author 2    | 2025-10-08 20:59:49 | region 2 | completed | discription 2    |
|    3 | title 3 | author 3    | 2025-10-08 21:00:19 | region 3 | completed | discription 3    |
|    4 | title 4 | author 4    | 2025-10-08 21:00:36 | region 4 | pending   | discription 4    |
|    5 | title 5 | author 5    | 2025-10-08 21:07:03 | region 5 | completed | discription 5    |
+------+---------+-------------+---------------------+----------+-----------+------------------+
5 rows in set (0.01 sec)
```

- To Edit the Patent ‘s title

```
--- PATENT MANAGEMENT SYSTEM ---
1. Look for a Patent
2. Show All Patents
3. Create Patent
4. Edit Patent
5.Delete patent
6. Exit
Choose any one option from above: 4

--- Edit Patent ---
Enter Seriel number to edit: 1
Current details:
(1, 'title 1', 'author 1', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'description
1')

1. Update Title
2. Update Author
3. Update Region
4. Update Status
5. Update Description
choose an option from above: 1
New title: Title 1
Updated successfully!
```

- To Edit the Patent ‘s author name

```
--- Edit Patent ---
Enter Seriel number to edit: 1
Current details:
(1, 'Title 1', 'author 1', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'description
1')

1. Update Title
2. Update Author
3. Update Region
4. Update Status
5. Update Description
choose an option from above: 2
New author: Author 2
Updated successfully!
```

- To Edit the Patent ‘s Region

```
--- Edit Patent ---
Enter Seriel number to edit: 1
Current details:
(1, 'Title 1', 'Author 2', datetime.datetime(2025, 10, 8, 20, 59, 20), 'reg 1', 'pending', 'descriptio
n1')

1. Update Title
2. Update Author
3. Update Region
4. Update Status
5. Update Description
choose an option from above: 3
New region: region 1
Updated successfully!
```

- To Edit the Patent ‘s Status

```
--- Edit Patent ---
Enter Seriel number to edit: 1
Current details:
(1, 'Title 1', 'Author 2', datetime.datetime(2025, 10, 8, 20, 59, 20), 'region 1', 'pending', 'descrip
tion1')

1. Update Title
2. Update Author
3. Update Region
4. Update Status
5. Update Description
choose an option from above: completed
Updated successfully!
```

- To Edit the Patent ‘s Discription

```
--- Edit Patent ---
Enter Seriel number to edit: 1
Current details:
(1, 'Title 1', 'Author 2', datetime.datetime(2025, 10, 8, 20, 59, 20), 'region 1', 'pending', 'descrip
tion1')

1. Update Title
2. Update Author
3. Update Region
4. Update Status
5. Update Description
choose an option from above: 5
New description: Discription 1
Updated successfully!
```

- To Delete a Patent Details

```
--- PATENT MANAGEMENT SYSTEM ---
1. Look for a Patent
2. Show All Patents
3. Create Patent
4. Edit Patent
5.Delete patent
6. Exit
Choose any one option from above: 5

--- Delete Patent ---
Enter Serial number to delete: 5
Are you sure you want to delete patent ? (y/n): y
Patent deleted successfully!
```

- After deleting the patent 5 Database

s_no	title	author_name	date_time	region	status	idea_description
1	Title 1	Author 2	2025-10-08 20:59:20	region 1	pending	Discription 1
2	title 2	author 2	2025-10-08 20:59:49	region 2	completed	discription 2
3	title 3	author 3	2025-10-08 21:00:19	region 3	completed	discription 3
4	title 4	author 4	2025-10-08 21:00:36	region 4	pending	discription 4

4 rows in set (0.01 sec)

- To close the Iteration and stop the program

```
--- PATENT MANAGEMENT SYSTEM ---
1. Look for a Patent
2. Show All Patents
3. Create Patent
4. Edit Patent
5.Delete patent
6. Exit
Choose any one option from above: 6
```

CONCLUSION

The Patent Manager project successfully demonstrates the integration of Python programming with a MySQL database to create a functional patent management system. It allows users to efficiently perform essential operations such as creating, searching, editing, displaying, and deleting patent records. The project implements robust user interaction through a menu-driven interface and ensures data integrity with well-managed database transactions.

This system can be effectively used by inventors, startups, or educational institutes to organize and maintain patents systematically. It not only provides easy access to patent information through multiple search criteria but also simplifies record maintenance by enabling updates and deletions as needed.

The modular design and clear workflow make the system user-friendly and adaptable for future enhancements such as adding search by keywords or integrating export features. Overall, this project offers practical exposure to database management concepts, CRUD operations, and software development lifecycle, making it an excellent learning tool for students.

FUTURE ENHANCEMENT

- **User Authentication and Role Management**
  Add login functionality with roles like admin and user to restrict certain operations like editing and deleting patents.
- **Keyword Search**
  Implement text-based search to find patents by keywords in titles or descriptions for more flexible querying.
- **Export and Import**
  Allow exporting patent records to CSV or Excel files and importing from such files for easy data exchange.
- **Improved User Interface**
  Create a graphical user interface (GUI) or web-based front end for better usability instead of command-line interaction.
- **Attachment Support**
  Enable storing related documents or images linked to each patent record in the database.
- **Pagination and Sorting in Display**
  Add pagination and advanced sorting/filtering in show-all and search results for better navigation through large data sets.
- **Backup and Restore**
  Build tools to backup the database and restore from backups to secure data.
- **Multi-language Support**
  Add support for multiple languages in the UI for wider accessibility.
- **Analytics Dashboard**
  Integrate charts and reports showing patent statistics and trends over time.

BIBLIOGRAPHY / REFERENCES

- MySQL Official Documentation, https://dev.mysql.com/doc/
- MySQL Connector/Python Developer Guide, https://dev.mysql.com/doc/connector-python/en/
- Python 3 Official Documentation, https://docs.python.org/3/
- W3Schools SQL Tutorial, https://www.w3schools.com/sql/
- CBSE Computer Science Project Guidelines, https://cbseacademic.nic.in/
- Various online forums and tutorials for Python-MySQL integration, such as GeeksforGeeks, Stack Overflow, and Real Python.
- "Patent Management Software Best Practices," Triangle IP blog, 2025.
- "Design and Implementation of a Patent Management System," Journal of Software Engineering, 2024.